

Ini adalah penjelasan mendalam mengenai kode C# yang Anda berikan. Kode ini berfungsi untuk melakukan proses **Binarization** (mengubah gambar menjadi murni Hitam dan Putih) menggunakan teknik **Direct Memory Access (LockBits)**.

Teknik ini jauh lebih cepat daripada menggunakan metode standar GetPixel dan SetPixel karena kita memanipulasi *byte* di memori secara langsung.

Berikut adalah penjelasan per bagian dan per baris:

1. Library dan Namespace

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing; // Inti pengolahan grafis
using System.Drawing.Imaging; // <- Penting: Untuk LockBits, BitmapData, ImageLockMode
using System.Runtime.InteropServices; // <- Penting: Untuk Marshal.Copy (Salin memori)
```

- **System.Drawing:** Menyediakan akses ke kelas Bitmap dan Graphics dasar.
- **System.Drawing.Imaging:** Menyediakan fungsionalitas lanjutan untuk memanipulasi format file gambar dan data piksel mentah (BitmapData).
- **System.Runtime.InteropServices:** Menyediakan koleksi kelas untuk mengakses memori yang tidak terkelola (*unmanaged memory*). Kita butuh ini untuk menyalin data dari pointer memori gambar ke dalam array C# biasa.

2. Struktur Kelas

C#

```
namespace MiniPhotoshop.Logic.ImageProcessing
{
    public static class BlackWhite
```

```
{
```

- **namespace:** Mengelompokkan kode agar rapi.
- **public static class BlackWhite:** Kelas dibuat static karena fungsi di dalamnya adalah fungsi utilitas (alat bantu) yang tidak menyimpan *state* atau data instan. Anda bisa memanggilnya langsung tanpa `new BlackWhite()`.

3. Fungsi ApplyBinarization (Pengatur Logic)

Fungsi ini bertugas menerjemahkan input pengguna (slider) menjadi nilai angka *threshold* (ambang batas).

C#

```
public static Bitmap ApplyBinarization(Bitmap originalImage, int sliderStep)
{
    int threshold;
    switch (sliderStep)
    {
        case 0: threshold = 50; break;
        case 1: threshold = 100; break;
        case 2: threshold = 127; break; // Nilai tengah standar
        case 3: threshold = 150; break;
        case 4: threshold = 200; break;
        default: threshold = 127; break;
    }
    // Panggil fungsi inti
    return ApplyThreshold(originalImage, threshold);
}
```

- **Logika Switch:** Mengubah langkah slider (0-4) menjadi nilai 0-255.
- **threshold:** Batas penentu. Jika kecerahan piksel di bawah nilai ini, jadi hitam. Jika di atas, jadi putih.
- **return ApplyThreshold(...):** Melempar tugas berat ke fungsi pemroses memori di bawahnya.

4. Fungsi ApplyThreshold (Inti Pemrosesan)

Ini adalah bagian paling penting. Kita akan bedah langkah demi langkah.

A. Persiapan Gambar

C#

```
private static Bitmap ApplyThreshold(Bitmap originalImage, int threshold)
{
    // Buat salinan gambar agar gambar asli tidak rusak
    Bitmap resultImage = new Bitmap(originalImage);

    // Tentukan area persegi panjang yang ingin kita kunci (seluruh gambar)
    Rectangle rect = new Rectangle(0, 0, resultImage.Width, resultImage.Height);
```

B. Mengunci Memori (LockBits)

C#

```
// Kunci bit gambar di memori sistem
BitmapData bmpData = resultImage.LockBits(rect, ImageLockMode.ReadWrite,
    resultImage.PixelFormat);
```

- **LockBits:** Memberi tahu Sistem Operasi: "Jangan pindahkan posisi memori gambar ini, saya mau mengedit isinya secara langsung."
- **ImageLockMode.ReadWrite:** Kita akan membaca data piksel (untuk menghitung grayscale) dan menulis data baru (hitam/putih).

C. Menyalin Data ke Array (Marshalling)

C#

```
// Ambil alamat memori pertama (pointer) dari data gambar
IntPtr ptr = bmpData.Scan0;
```

```
// Hitung total bytes. Stride adalah lebar baris (termasuk padding memori)
```

```
int bytes = Math.Abs(bmpData.Stride) * resultImage.Height;
```

```
// Siapkan array kosong untuk menampung data
```

```
byte[] rgbValues = new byte[bytes];
```

```
// SALIN data dari memori tidak terkelola (ptr) ke array C# (rgbValues)
```

```
Marshal.Copy(ptr, rgbValues, 0, bytes);
```

- **ptr**: Penunjuk alamat memori awal gambar.
- **Stride**: Lebar satu baris gambar dalam *byte*. Seringkali lebar *byte* tidak sama persis dengan lebar piksel karena komputer menambahkan "padding" agar data rapi di memori.
- **Marshal.Copy**: Ini kuncinya. Kita memindahkan data dari "dunia pointer" yang rumit ke "dunia array" C# yang mudah diolah.

D. Loop Pemrosesan (Super Cepat)

C#

```
int bytesPerPixel = Image.GetPixelFormatSize(resultImage.PixelFormat) / 8;
```

```
int stride = bmpData.Stride;
```

```
// Loop vertikal (Baris)
```

```
for (int y = 0; y < resultImage.Height; y++)
```

```
{
```

```
    // Hitung posisi awal baris saat ini
```

```
    int rowOffset = y * stride;
```

```
    // Loop horizontal (Kolom)
```

```
    for (int x = 0; x < resultImage.Width; x++)
```

```
    {
```

```
        // Hitung posisi tepat piksel saat ini dalam array 1 dimensi
```

```
        int i = rowOffset + (x * bytesPerPixel);
```

- Kita menggunakan array 1 dimensi untuk merepresentasikan gambar 2 dimensi. Rumus i di atas adalah cara kita menemukan koordinat (x, y) di dalam array datar tersebut.

E. Rumus Grayscale & Thresholding

C#

```
// 1. Grayscale
// Urutan memori Windows Bitmap biasanya BGR (Blue, Green, Red), bukan RGB.
// i = Blue, i+1 = Green, i+2 = Red
int gray = (int)(rgbValues[i + 2] * 0.299 + // Red
                rgbValues[i + 1] * 0.587 + // Green
                rgbValues[i] * 0.114);    // Blue
```

- Kita menggunakan rumus Luminosity standar untuk mengubah warna menjadi abu-abu agar sesuai dengan persepsi mata manusia:

$$\text{Gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

- Perhatikan indeks array: i biasanya adalah komponen Blue, i+1 Green, dan i+2 Red.

C#

```
// 2. Threshold (Hitam Putih)
// Jika abu-abu < batas, set 0 (Hitam). Jika tidak, set 255 (Putih).
byte bwValue = (gray < threshold) ? (byte)0 : (byte)255;

// Masukkan nilai bwValue ke ketiga channel warna agar jadi grayscale murni
rgbValues[i] = bwValue; // Blue
rgbValues[i + 1] = bwValue; // Green
rgbValues[i + 2] = bwValue; // Red
}
```

- Di sini kita menimpa nilai warna asli dengan nilai baru (0 atau 255).

F. Membersihkan & Menyimpan

C#

```
// Salin data yang sudah dimodifikasi dari array KEMBALI ke pointer memori gambar
Marshal.Copy(rgbValues, 0, ptr, bytes);
```

```
// Lepaskan kunci memori. Sangat penting agar tidak memory leak/error.  
resultImage.UnlockBits(bmpData);  
  
return resultImage;  
}
```

- Kita mengembalikan data dari array ke memori gambar asli, lalu membuka kuncinya (UnlockBits) agar gambar bisa digunakan lagi oleh Windows/UI.

Ringkasan Alur Data

1. **Image (UI)** -> Masuk ke fungsi.
2. **LockBits** -> Mengunci gambar di RAM.
3. **Marshal.Copy (In)** -> Memindah data piksel ke Array byte[].
4. **Looping** -> Membaca BGR, hitung rata-rata, tentukan hitam/putih, tulis ulang ke Array.
5. **Marshal.Copy (Out)** -> Mengembalikan Array ke memori gambar.
6. **UnlockBits** -> Selesai.

Apakah Anda ingin saya menjelaskan lebih lanjut tentang bagaimana cara menangani format gambar yang memiliki *Alpha Channel* (Transparansi/PNG) dengan kode ini?